



David Darras

Date de création : 29/05/2024
Dernière modification : 13/10/2024

Préambule

- **OCaml** (*Objective Categorical Abstract Machine Language*) : Langage de programmation fonctionnel orienté objet. Un **langage fonctionnel** traite les fonctions comme des valeurs, permettant ainsi de les utiliser comme paramètres ou résultats d'autres fonctions. Il est généralement utilisé pour éviter les effets de bord.
- **Inférence de types** : Le type des expressions est déduit automatiquement à la compilation. Le programmeur n'a pas besoin de spécifier explicitement le type, contrairement au C.
- **Polymorphisme paramétrique** : Il est possible d'écrire des fonctions génériques en utilisant des types polymorphes comme 'a, ce qui signifie qu'une fonction peut traiter différents types de données sans connaître leur structure exacte. Par exemple, on peut créer des listes d'arbres.
- **Mécanisme d'exceptions** : Il permet d'interrompre l'exécution normale d'un programme et de la reprendre à un endroit spécifique prévu pour gérer les situations exceptionnelles, comme une division par zéro.
- **Portabilité** : Il est possible de produire un bytecode, comme en Java, et de l'interpréter avec une machine virtuelle, ou bien de compiler notre code en un exécutable, comme en C.
- **Garbage collector** : Comme en Java, vous n'avez pas besoin de vous soucier de la gestion de la mémoire ; tout est géré de manière automatique et sûre.
- **Toplevel** (ou REPL) : Environnement interactif qui permet d'évaluer des expressions OCaml et d'exécuter des instructions directement dans le terminal. On termine une expression par ;;

Évaluation d'une expression

```
(* Expressions *)
# 8 / 3;;
- : int = 2

# 1 > 0;;
- : bool = true

# sqrt 9.;;
- : float = 3.
(* Sortie => Nom : Type = Résultat *)
(* - : désigne quelque chose d'anonyme *)
```

Les variables constantes

```
# let x = 13;; (* const int x = 13; *)
val x : int = 13

# let y = x - 1;; (* const int y = x - 1; *)
val y : int = 12

# let res =
  let a = 1 in (*a=1*)
  let b = 2 + a in (*b=2+a=2+1=3*)
  let c = 3 * b + a in (*c=3*b+a=3*3+1=10*)
  c / 2;; (*res=c/2=10/2=5*)
val res : int = 5

# let x = 2 in
  [ 6; x + 4; 1; x - 1 ];;
- : int list = [6; 6; 1; 1]
(* La valeur retournée n'est pas stockée *)

(* Définition multiple *)
# let x = 1 and y = 2 and z = 3;;
val x : int = 1
val y : int = 2
val z : int = 3

(* Tuple *)
# let author = ("David", 21);;
val author : string * int = ("David", 21)
(* "*" fait référence au produit cartésien *)

(* Récupérer les valeurs du tuple *)
```

```
# let (name, age) = author;;
val name : string = "David"
val age : int = 21
```

Remarques :

- let <var> = <expr1> permet de créer une variable non mutable. Il est donc impossible d'écrire <var> = <expr2> ensuite. Mais on peut écraser la définition précédente avec let <var> = <expr2>.
- let <var> = <expr1> in <expr2> : <expr1> produit un résultat qu'on stocke dans <var>, puis cette variable évaluée est utilisée dans <expr2>.

Les variables mutables

```
# let x = ref 6;;
val x : int ref = {contents = 6}

# x;;
- : int ref = {contents = 6}

# !x;; (* x.contents;; *)
- : int = 6

(* x.contents <- x.contents + 1;; *)
# x := !x + 1 ;;
- : unit = ()

# !x;; (* x.contents;; *)
- : int = 7

(* Définition *)
# type 'a ref = { mutable contents : 'a }
  let ref x = { contents = x }
  let (!) r = r.contents
  let (:=) r x = r.contents <- x
;;
```

Les opérateurs

- Pour les entiers, on utilise les opérateurs habituels : +, -, *, /, mod.
- Pour les flottants, on rajoute un point après l'opérateur : +., -., *., /., mod_float. De plus, ** permet de calculer une puissance.
- On peut ajouter un _ dans une séquence de chiffres pour améliorer la lisibilité.

```
# 8 - 1;; (* opérateur infixe *)
- : int = 7

# (-) 8 1;; (* opérateur préfixe *)
- : int = 7

# 3.5 +. 6.;;
- : float = 9.5

# 10_000 + 11;;
- : int = 10011
```

```
(* Définir un opérateur *)
# let (+!) (a,b) (c,d) = (a+c,b+d) ;;
val (+!) : int * int -> int * int -> int * int = <fun>

# let res = (1,2) +! (3,4) ;;
val res : int * int = (4, 6)

# let (>) x y = (x,y) ;;
val (>) : 'a -> 'b -> 'a * 'b = <fun>

# 2 > 3 ;;
- : int * int = (2, 3)

(* Opérateur d'application inverse *)
# let (|>) x f = f x;;
val (|>) : 'a -> ('a -> 'b) -> 'b = <fun>

# 1 |> (fun x -> x + 1) |> (fun y -> (2*y, y+1)) ;;
- : int * int = (4, 3)
```

Instructions conditionnelles

```
# let p x = (* const char* p(int x) { *)
  if x = 1 then (* if (x == 1) *)
    "A" (* return "A"; *)
  else if x < 3 then (* else if (x != 3) *)
    "B" (* return "B"; *)
  else if x > 0 then (* else if (x > 0) *)
    "C" (* return "C"; *)
  else (* else *)
    "D"; (* return "D"; } *)
val p : int -> string = <fun>

# let p x y =
  if x + 1 < 0 && y then (* ET *)
    (* Définir un bloc d'instructions *)
    begin (* { *)
      print_string "Hello! ";
      print_string "How are you?";
      0
    end (* } *)
  else if x * 2 > 0 || not y then (* OU *)
    1
  else
    -1;;
val p : int -> bool -> int = <fun>

(* Pattern matching *)
# let p x =
  match x with (* switch(x) { *)
    0 -> "A" (* case 0: return "A"; *)
  | 1 -> "B" (* case 1: return "B"; *)
  | 13 -> "C" (* case 13: *)
  | 14 -> "D" (* case 14: return "D"; *)
  | _ -> "D"; (* default: return "D"; } *)
val p : int -> string = <fun>
```

Les types

L'inférence de type permet à OCaml de déduire le type de notre expression sans que l'on ait à lui fournir un type. Si on a if x then ..., on déduit que x est un booléen, si on

à $x + y$ on sait que x et y sont des entiers, etc. Dans le cas où le type n'est pas évident, on utilise un type générique 'a', 'b', 'c', etc. Mais il est aussi possible de spécifier le type d'une variable.

```
(* Inférence de type *)
# let sum_if_true test v1 v2 =
  ( if test v1 then v1 else 0 )
  + ( if test v2 then v2 else 0 );;
val sum_if_true : (int -> bool)
  -> int -> int -> int = <fun>

(* Type générique *)
# let foo test x y = if test x then x else y;;
val foo : ('a -> bool) -> 'a -> 'a -> 'a = <fun>

(* Cast *)
(* Convertir un float en int *)
# let ratio x y = Int.to_float x /. Float.of_int y;;
val ratio : int -> int -> float = <fun>

(* Imposer un type *)
# let square (x:int) : string = Int.to_string (x*x);;
val square : int -> string = <fun>

# square 3 ;;
- : string = "9"
```

Les fonctions

Contrairement au C, il ne faut pas appeler une fonction comme ceci : $f(x,y,z)$ car (x,y,z) représente un tuple. Il faut utiliser la même syntaxe que dans un shell : $f\ x\ y\ z$ ou bien $(f\ (x)\ (y)\ (z))$ si vous avez besoin de parenthèses, par exemple si $x = -1$ et $y = g\ 2\ 4$.

```
(* int mul3(int x, int y) { return x * y; } *)
(* Forme simplifiée *)
# let mul3 x y = x * y;;
(* Forme curriifiée *)
# let mul3 = fun x -> fun y -> x * y;;
(* Sortie => Nom : param1 -> param2 -> res = <fun> *)
val mul3 : int -> int -> int = <fun>

(* Fonction anonyme *)
# (fun x -> x ** 2.) 9.;;
- : float = 81.
(* Équivalent à la fonction lambda en C++ *)
(* ([](float x){ return x * x; })(9.0f) *)

(* Fonction récursive *)
# let rec factorial n = match n with
  0 | 1 -> 1
  | n -> factorial (n-1);;

# let rec factorial = function
  0 | 1 -> 1
  | n -> factorial (n-1);;
(* Ne pas oublier le "rec" ! *)
(* function = fun n -> match n with *)

(* Fonction récursive terminale *)
# let factorial n =
  let rec wrapper n res = match n with
```

```
  0 | 1 -> res
  | n -> wrapper (n-1) (n*res)
  in wrapper n 1;;
val factorial : int -> int = <fun>

(* Argument libellé *)
# let div ~num ~denom = float num /. float denom;;
val div : num:int -> denom:int -> float = <fun>

# div ~denom:10 ~num:3 ;;
- : float = 0.3

# let num = 3 in
  let denom = 4 in
  div ~num ~denom;;
- : float = 0.75

(* Argument optionnel *)
# let concat ?sep x y =
  let sep = match sep with
    None -> ""
    | Some x -> x
  in x ^ sep ^ y ;;

# concat "foo" "bar" ;;
- : string = "foobar"

# concat ~sep:":" "foo" "bar" ;;
# concat ?sep:(Some ":") "foo" "bar";;
- : string = "foo:bar";;

(* Récursions mutuelles *)
# let rec isEven x =
  match x with
  0 -> true
  | _ -> isOdd (x - 1)
and isOdd x =
  match x with
  0 -> false
  | _ -> isEven (x - 1);;

(* Ordre supérieur *)
(* Une fonction peut recevoir/produire une fonction *)
# let add1 x = (+) x 1;;
val add1 : int -> int = <fun>

(* (f o g)(x) *)
# let compose f g x = f (g x);;
val compose : ('a -> 'b) -> ('c -> 'a) -> 'c -> 'b = <fun>
```

Les listes

```
(* Définition *)
# let l1 = [1; 2; 3];;
val l1 : int list = [1; 2; 3]

# let l2 = ["a"; 2; 3.];; (* FAUX *)
(* Impossible de mélanger les types comme en Python *)

# let l3 = [1; 2; 3];;
val l3 : (int * int * int) list = [(1, 2, 3)]
(* Ce sont des points-virgules, pas des virgules ! *)
(* Les virgules donnent un tuple *)
```

```
# let l = ["a"; "bc"];;
val l : string list = ["a"; "bc"]

# List.map String.length l;;
- : int list = [1; 2]
(* Applique la fonction à tous les éléments *)

(* Ajouter un élément en tête d'une liste *)
# let l2 = "x" :: "y" :: l;;
val l2 : string list = ["x"; "y"; "a"; "bc"]

# let l3 = l :: ["@"];; (* FAUX *)
(* On ne peut pas ajouter d'éléments en fin de liste *)

(* Concaténation - moins efficace *)
# let l4 = (l :: []) @ [2] @ [3; 4];;
val l4 : int list = [1; 2; 3; 4]

(* Utilisation des matchs *)
# let get_head (head :: tail) = head;;
(* Remarque : get_head [] ne fonctionne pas *)

# let rec length list = match list with
| [] -> 0
| head :: tail -> 1 + (length tail);;
```

Tableaux et itérations

```
(* Définition *)
# let tab = [|1; 2; 3|];;
val tab : int array = [|1; 2; 3|]

(* Modifier une valeur *)
# tab.(2) <- 9;; (* tab[2] = 9; *)
- : unit = () (* Fonction à effet de bords *)

# tab;;
- : int array = [|1; 2; 9|]

(* Taille *)
# Array.length tab;;
- : int = 3

(* Itération *)
# let i = ref 0;;
val i : int ref = {contents = 0}

# let res = ref 1;;
val res : int ref = {contents = 1}

# while (!i < 8) do
  res := 2 * !res;
  i := !i + 1;
done;;
- : unit = ()

# res;;
- : int ref = {contents = 256}

# let t = [|0; 0; 0|];;
val t : int array = [|0; 0; 0|]
```

```
# for i = 0 to (Array.length t) - 1 do
  t.(i) <- i;
done;;

(* Il existe aussi downto *)
```

Variants - Types somme

```
(* Equivalent à une énumération en C *)
# type phone = Apple | Samsung | Huawei | Wiko;;
type phone = Apple | Samsung | Huawei | Wiko
(* typedef enum { Apple, Samsung, Huawei, Wiko } phone; *)

# let price p = match p with
  Apple -> 2500
| Samsung -> 1200
| _ -> 400;;
val price : phone -> int = <fun>

(* Equivalent à union en C *)
# type op =
  Add of int * int
| Neg of int;;
type op = Add of int * int | Neg of int

# let eval op = match op with
  Add(a,b) -> a + b
| Neg(x) -> (-1) * x;;

# eval (Add(1,2));;
- : int = 3

(* Type de données récursive *)
# type 'a myList =
  Nil (* Liste vide *)
| Cons of 'a * 'a myList
      (* tête, queue *)
# Cons(1, Cons(2, Nil));;
- : int myList = Cons (1, Cons (2, Nil))

# type tree =
  (* Feuille vide *)
  V
| N of tree * int * tree ;;
      (* Noeud - fils gauche, étiquette, fils droit *)
type tree = V | N of tree * int * tree

# let rec nodes t = match t with
  V -> 0
| N(g,_,d) -> 1 + (nodes g) + (nodes d);;
(* _ indique qu'on ne veut pas récupérer la valeur du paramètre *)

# let abr = N(N(V,1,V),2,N(N(V,3,V),4,V));;
val abr : tree = N(N(V, 1, V), 2, N(N(V, 3, V), 4, V))
```

Records

```
(* Similaire aux structures en C *)
(* typedef struct { const float x, y; } cpoint; *)
# type cpoint = { x : float; y : float };;
type cpoint = { x : float; y : float }
```

```

(* avec x et y constants *)

(* typedef struct { float x, y; } point; *)
# type point = { mutable x : float; mutable y : float };
type point = { mutable x : float; mutable y : float }
(* avec x et y modifiables *)

# let magnitude {x=x';y=y'} = sqrt (x'**2.+y'**2.);
val magnitude : point -> float = <fun>

# type rectangle = { pmin : point; pmax : point };
# type circle = { pcenter : point; radius : float };

(* Equivalent à union en C *)
# type form =
  | Rectangle of rectangle
  | Circle of circle;;
(* Attention, les tags commencent par une majuscule ! *)

# let surface f = match f with
  | Rectangle { pmin; pmax } ->
    (pmax.x -. pmin.x) *. (pmax.y -. pmin.y)
  | Circle { pcenter; radius } ->
    3.1415926 *. radius ** 2.;

# let square = Rectangle { pmin = {x = 0.; y = 0.};
                          pmax = {x = 10.; y = 10.} };

```

Option

```

# let div x y = match y with
  | 0 -> None
  | _ -> Some(x / y);
val div : int -> int -> int option = <fun>

# let a = div 4 2;;
val a : int option = Some 2

# let b = div 1 0;;
val b : int option = None

```

Erreurs et Exceptions

Une erreur intervient lors de la compilation, tandis qu'une exception se produit lors de l'exécution.

```

# let div x = x / 0;;
val div : int -> int = <fun>

# div 3;;
Exception: Division_by_zero.

# let x = 1 + "2";;
Error: This expression has type string but
an expression was expected of type int.

# let tail = function
  _ :: t -> t
  | _ -> failwith "No tail";
(* failwith = lève une exception *)

(* Créer ses propres exceptions *)
exception Negative of int;;
exception Null;;

let getNum x =
  if x < 0 then
    raise (Negative x) (* lève une exception *)
  else if x = 0 then
    raise Null
  else (* x > 0 *)
    x;;

let () =
  try
    let res = getNum (-10) in
    print_string ("Nombre positif : "
      ^ string_of_int res)
  with
  | Negative n ->
    print_string ("Erreur : Nombre négatif : "
      ^ Int.to_string n)
  | Null -> print_string "Erreur : Nombre nul";

```